A Comparative Security Evaluation of AI-Generated Code Across Large Language Models and Programming Languages

Deniz Aydın[a,b,*], Şerif Bahtiyar[a]

[a]Department of Computer Engineering, Istanbul Technical University, Maslak, Istanbul, 34469, Türkiye
[b]Research & Development, Siemens, Kartal Esentepe, Istanbul, 34870, Türkiye

ARTICLE INFO

Keywords:
AI-generated Code
Large Language Model
Vulnerability Analysis
CWE
CVSS
Secure Code Generation

ABSTRACT

Large Language Models (LLMs) have been widely used in software development, yet the security of AI-generated code remains a critical concern. This research examines security vulnerabilities in code generated by seven LLMs, which are GPT-5, Claude Sonnet 4.5, DeepSeek R1 70B, Gemini 3 Flash, Llama 3.1 405B, Mistral Large 2, and Nova Pro, across four programming languages, which are JavaScript, Python, C++, and Java. We propose a new approach using 100 complex prompts per language identical across the LLMs. We analyzed the most frequent Common Weakness Enumeration (CWE) categories across different LLMs and programming languages. Each snippet was analyzed via a hybrid pipeline that combines three Static Application Security Testing (SAST) tools, which are Semgrep, Snyk and SonarCloud, with manual review. Findings were classified by Common Vulnerability Scoring System (CVSS) severity. The results show that 936 of the 2,800 snippets contain vulnerabilities, and we identified 1,703 vulnerabilities from 36 CWEs, unevenly distributed across languages (JavaScript: 695, Python: 398, Java: 348, C++: 262). Although all LLMs introduce vulnerabilities, their CWE and severity distributions vary significantly across both LLMs and programming languages, with the same model concentrating on different CWE subsets depending on the language. Because raw vulnerabilities ignore differences in code size and vulnerability severity, we introduce two new metrics for a fair comparison, namely Vulnerabilities per Line of Code (V/LoC) and Weighted Security Risk per Line of Code (WSR/LoC). Our research highlights the need for language aware and model aware security measures to mitigate risks in AI-generated code.

1. Introduction

Large Language Models (LLMs) have been widely adopted in software development, enabling developers to generate code efficiently. A recent survey reports that more than 97% of software developers used artificial intelligence (AI) coding tools in their work at some point [10]. However, security risks of AI-generated code are still a critical concern. Studies have shown that LLMs frequently produce security vulnerabilities in generated code [2, 4, 7, 14, 15, 22, 26, 29, 31, 35, 44, 45, 50].

In this research, we examine the security risks of AI-generated code by analyzing the frequency, severity, and Common Weakness Enumeration (CWE) distribution of vulnerabilities across different LLMs and different programming languages. We evaluated seven commonly used LLMs, which were developed by different companies, namely GPT-5 [30], Claude Sonnet 4.5 [5], DeepSeek R1 70B [11], Gemini 3 Flash [16], Llama 3.1 405B [27], Mistral Large 2 [28], and Nova Pro [3]. The evaluation was conducted on four programming languages, which are JavaScript, Python, C++, and Java. For each LLM and each programming language, we generated code snippets from 100 identical prompts within each programming language, totally 2,800 AI-generated code snippets.

The generated outputs were analyzed using a two-layered security assessment approach by combining Static Application Security Testing (SAST) tools, which are Semgrep [36], Snyk [39], and SonarCloud [40] with manual code reviews to identify and classify vulnerabilities across different CWE categories [43]. With this approach, a standardized assessment is provided to compare the security performance of the LLMs to identify potential risks in AI-generated code.

We introduce two evaluation metrics to provide an accurate comparison between different LLMs, namely Vulnerabilities per Line of Code (V/LoC) and Weighted Security Risk per Line of Code (WSR/LoC). We use V/LoC to compare the security of different LLMs because the number of vulnerabilities alone may be misleading due to variations in code size. For instance, Claude Sonnet 4.5's generated JavaScript code snippets are approximately seven times longer than Nova Pro's generated ones, which makes direct vulnerability comparisons inaccurate. Similarly, WSR/LoC is introduced to address the varying severity of vulnerabilities between different LLMs, as some models generate critical vulnerabilities more frequently than other models. By incorporating severity-based weighting, WSR/LoC enables a more accurate and fair assessment of security risks across different LLMs.

The primary purpose of this work is to provide a systematic security evaluation of LLM generated code in multiple programming languages. Most of the previous research has focused on a single programming language or a small number of models. Such approaches do not capture differences

*Corresponding author
aydinde18@itu.edu.tr (D. Aydın); bahtiyars@itu.edu.tr (Ş. Bahtiyar)
ORCID(s):

Preprint not peer reviewed
This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

in code length, level of details, or severity of detected vulnerabilities. This circumstance makes it difficult to compare different LLMs in a meaningful way. In this paper, our aim is to address this gap by analyzing seven LLMs and four programming languages under the same set of prompts. Moreover, we introduce two new metrics to efficiently compare different LLMs' security capabilities. Beyond raw vulnerability counts, we provide a more practical view of the security risks that LLM-generated code can introduce into a software project. The main contributions of this research are as follows.

- A systematic evaluation of security risks in AI-generated code in seven LLMs and four programming languages. The same set of 100 prompts is given to all seven LLMs within each programming language, resulting in 700 code snippets per language and 2,800 code snippets in total. Each snippet is analyzed using a two-layered approach that combines SAST tools and manual code review. Vulnerabilities are mapped to CWE categories and classified according to their CVSS severity levels that enable us to construct valuable statistical data which give insights about security vulnerabilities in AI generated code.
- Two new evaluation metrics, V/LoC and WSR/LoC. V/LoC normalizes the number of vulnerabilities by the size of the generated code, and WSR/LoC takes into account the severity of each vulnerability. The metrics enable a fair comparison between LLMs that produce different amounts of code with different severity profiles. Thus, it makes the relative security posture of each model easier to interpret and compare.

The remainder of the paper is organized as follows. Section 2 discusses the state of the art. Section 3 describes the proposed methodology. The experimental results are given in Section 4. Section 5 concludes the paper.

2. State of the Art in AI-Generated Code Security

2.1. Security Vulnerabilities in AI-Generated Code

Several studies have systematically evaluated security vulnerabilities present in AI-generated code, highlighting the security risks of Large Language Models. The state of the art has shown that AI-generated code frequently contains exploitable vulnerabilities. One of the earliest research, Fu et al. [14] analyzed 452 code snippets produced by GitHub Copilot and found that 29.5% of Python and 24.2% of JavaScript snippets contain vulnerabilities across 43 different CWE categories. In another research, Tihanyi et al. [45] demonstrated that 51.24% of 112,000 ChatGPT-3.5 generated C programs contained vulnerabilities. Later, Tihanyi et al. [44] improved this research by including 9 other LLMs which resulted in 62.07% of the generated programs being vulnerable. In another research, Pearce et al. [31] found that approximately 40% of programs are discovered to be vulnerable. A recent example shows that various programming languages with two different LLMs were analyzed by Ambati et al., where their analysis showed that 49% to 55.5% of samples include at least one vulnerability depending on the LLM [4]. Beyond per snippet evaluations, a large scale analysis of public GitHub repositories conducted by Schreiber and Tippe [35] examined 7,703 files attributed to four major AI tools, including ChatGPT, GitHub Copilot, Amazon CodeWhisperer and Tabnine, where the authors identified 4,241 CWE instances spanning 77 distinct CWEs. In another study, Li et al. [25] proposed SafeGenBench, an automatic evaluation framework that combines SAST tools and LLMs to assess the security vulnerabilities in AI generated code, reporting that the average overall accuracy across evaluated models is 37.44%. Other research studies also demonstrated the risk of using LLMs introducing vulnerabilities in generated code [2, 7, 15, 21, 22, 26, 29, 50]. A comparison of vulnerabilities of AI-Generated code is shown in Table 1.

2.2. Comparative Analysis Between AI-Generated and Human-Written Code

Many research studies not only investigated security risks in AI-generated code but also conducted comparative analysis between AI-generated and human written code to evaluate their relative security implications. One such example is the research conducted by Hamer et al. [18], which demonstrated that ChatGPT generated code produces 20% fewer vulnerabilities than human generated code. Another similar research by Asare et al. [6] claims that AI is less likely to introduce vulnerabilities in code than humans. Similar experiments have been conducted by Sandoval et al. [34] which concluded that the usage of LLMs does not increase the incidence rate. Another important aspect is examined in Perry et al. [32] which shows that developers who have access to AI tools generated less secure code than developers coding without AI assistance where the AI assisted group exhibited increased confidence in the security of their code, leading to overlooked risks and improper security practices. Another research with conflicting conclusion conducted by Siddiq et al. [38] showing that LLMs introduce a significant amount of code smells.

2.3. Prompt Engineering for Secure Code Generation

Apart from studies on security of AI generated code and its comparison to human written code, several recent studies have focused on whether security aware prompting or iterative refinement of prompts can improve the security of LLM generated code. The study by Shukla et al. [37] reported that successive prompt driven refinements of AI-generated code weakens the security of the resulting program despite the explicit intent of improving the program, raising serious concerns on the reliability of human AI iterative development pipelines for security critical software. In contrast, Yan et al. [48] investigated whether targeted feedbacks can guide

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

Table 1
Comparison of related studies on vulnerabilities in AI-generated code.

Research	LLM	Programming Language	Vulnerability Rate
[14] (2025)	GitHub Copilot	Python, JavaScript	29.5% (Python), 24.2% (JavaScript) snippets vulnerable
[45] (2023)	ChatGPT-3.5	C	51.24% of 112,000 programs vulnerable
[44] (2024)	9 LLMs	C	62.07% of generated programs vulnerable
[31] (2022)	GitHub Copilot	Python, C	39% Python, 50% C of programs vulnerable
[4] (2024)	2 LLMs	Multiple	49%–55.5% of samples contain at least one vulnerability
[22] (2023)	ChatGPT	C, C++, Python, HTML, Java	5/21 programs secure on first attempt (~76% vulnerable)
[26] (2024)	GitHub Copilot (newer versions)	Python	27.25%–36.54% of suggestions vulnerable
[35] (2025)	ChatGPT, Copilot, Amazon CodeWhisperer, Tabnine	Multiple (real GitHub repositories)	4,241 CWE instances across 7,703 files (77 distinct CWEs)
[7] (2025)	ChatGPT-4o, Claude 3.5 Sonnet, DeepSeek R1 70B, Llama 3.1 405B, Mistral Large 2, Nova Pro	JavaScript	275/600 (45.8%) snippets vulnerable; 602 vulnerabilities across 28 CWEs
[21] (2026)	Claude 3.5, Gemini 1.5, Codestral, GPT-4o, Llama 3	Python, Java, C++, C	284 vulnerabilities (7.1%) and 1,304 security hotspots (32.6%) across 4,000 samples; combined 1,588 (39.7%)
This work	GPT-5, Claude Sonnet 4.5, DeepSeek R1 70B, Gemini 3 Flash, Llama 3.1 405B, Mistral Large 2, Amazon Nova Pro	JavaScript, Python, C++, Java	936/2,800 (33.4%) snippets vulnerable; 1,703 vulnerabilities across 36 CWEs

LLMs to mitigate vulnerabilities in their own outputs, and reported that a single iteration of the Recursive Criticism and Improvement fixes 24.5% (GPT-3.5), 49.5% (GPT-4o-mini), and 64.7% (GPT-4o) of vulnerable snippets across multiple CWEs, showing that the success of iterative approaches strongly relies on the structure of the feedback loop and the underlying LLM. Another work by Bruni et al. [9] showed that prompt engineering techniques for secure code generation, where security aware prefixes such as instructing the model to behave as a security conscious developer reduced vulnerable code generation by 47% for GPT-4o-mini and 56% for GPT-4o without modifications to the underlying LLMs. Lastly, a prompt engineering study by Aldosari and Aldawsari [1] demonstrated that carefully crafted prompt templates can mitigate vulnerabilities across multiple LLMs and programming languages, although the authors also noted that even with explicit security aware prompts, LLMs may introduce new vulnerabilities while fixing obvious ones, indicating the need for new and hybrid mitigation strategies.

2.4. LLM Capability in Vulnerability Detection and Repair

Several recent research studies have examined security implications of AI-generated code focusing on the assessment of LLMs in secure coding tasks. A comprehensive analysis by Ullah et al. [46] where various LLMs were analyzed for vulnerability evaluation reached the conclusion that LLM models are not yet capable for doing security evaluations. Extending this work, Lekssays et al. [23] proposed LLMxCPG, a vulnerability detection framework that integrates Code Property Graphs with LLM reasoning, demonstrating that combining structural code representations with LLMs improves precision in detecting vulnerabilities compared with prompting LLMs on source code alone. Similarly, the CVE-Bench benchmark introduced by Wang et al. [47] evaluates LLM based software vulnerability detection through a four stage pipeline, where the results showed that current state of art LLMs struggle to consistently produce semantically correct security patches. A complementary study by Belozerov et al. [8] focuses on the full secure coding lifecycle from detection to repair, where the authors demonstrated that 2025 introduced LLMs such as GPT-4.1, GPT-5, and Claude Opus 4.1 reached 78% at overall detection and fix accuracy, indicating that more security capable models are started to be introduced. Other studies in this area include [13, 19, 20, 24, 41, 42].

2.5. Datasets and Benchmarks for Security Evaluation

Security evaluations of programs have been analyzed from many dimensions. For example, LLMs' capacity to be used in security related tasks have been examined by Rydén et al. [33] where the authors created LLMSecCode dataset to assess the security of code generated by LLMs. Hajipour et al. [17] also created a similar dataset named CodeLMSec to evaluate vulnerabilities in Black-Box Code Language Models. A research with similar goal has performed by Yildirim et al. [49] to evaluate different LLMs' performance for detecting API vulnerabilities. In a related

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

work, Dubniczky et al. [12] introduced the CASTLE benchmark, consisting of 250 micro-benchmark programs covering 25 common CWEs, where the authors evaluated 13 LLMs, 10 static analyzers, and 2 formal verification tools and demonstrated that LLMs perform exceptionally well at identifying vulnerabilities in small code snippets but their success declines and hallucination rates increase as the code size grows. All of these studies show that metrics are needed to compare vulnerabilities between human generated codes and AI generated codes.

2.6. Research Gap and Motivation

Overall, prior work consistently showed that AI-generated code can contain exploitable vulnerabilities, yet reported vulnerability rates vary substantially depending on the programming language, prompt design and evaluated LLM. Existing studies have commonly relied on raw vulnerability counts, binary vulnerable/non-vulnerable classifications or benchmark-specific detection accuracy. However, limited work has provided a controlled, multi-model comparison of AI-generated code while jointly considering CWE distribution, severity levels, vulnerability density, and severity-weighted risk. Therefore, this study addresses this gap through two contributions: first, by conducting a statistical evaluation of seven LLMs across four programming languages, and second, by proposing comparative metrics that go beyond raw vulnerability counts and account for the severity of vulnerabilities and the vulnerability density of code, thus enabling a fairer and more meaningful comparison of the security posture of different LLMs.

3. Methodology

3.1. Overview and study design

We adopt a controlled, factorial design: seven LLMs × four programming languages × 100 prompts, yielding 700 snippets per language and 2,800 snippets in total. Every model receives the same prompt text for a given task, and the only experimental variable in each cell of the design is the LLM. Output is collected with default decoding parameters and without any security-aware system prefix, so the resulting code reflects each model's baseline behaviour. The high-level study model is shown in Figure 1, and a detailed view of the analysis pipeline is given in Figure 2.

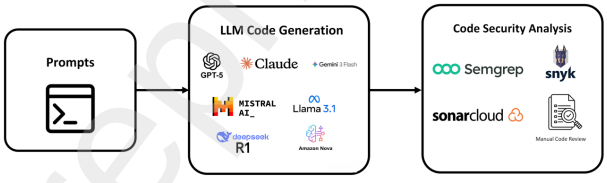


Figure 1: Proposed model for security evaluation of AI-generated code.

Table 2
Evaluated Large Language Models.

Model	Provider	Context
GPT-5	OpenAI	400k
Claude Sonnet 4.5	Anthropic	200k–1M
Gemini 3 Flash	Google	1M
DeepSeek R1 70B	DeepSeek	128k
Llama 3.1 405B	Meta	128k
Mistral Large 2	Mistral AI	128k
Amazon Nova Pro	Amazon	300k

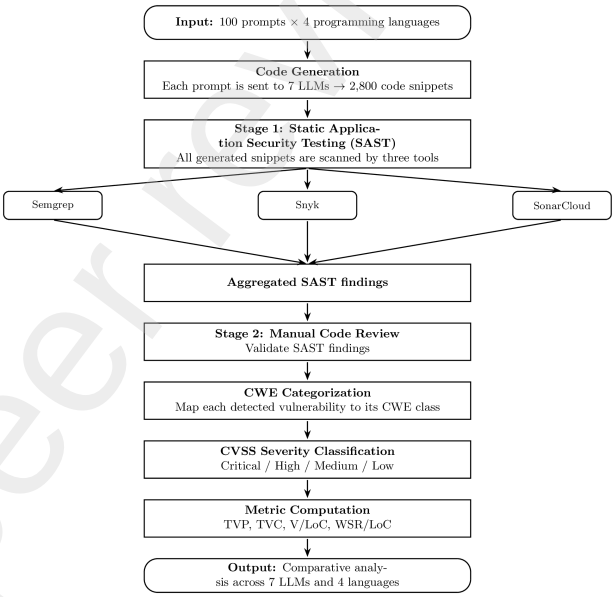


Figure 2: Two-stage hybrid analysis pipeline. Each generated snippet is scanned with three SAST tools and then manually reviewed before findings are mapped to CWE and CVSS.

3.2. LLM selection

The seven LLMs were chosen to cover different providers, training pipelines and parameter scales while being widely used in production. Each LLM is developed by a different organization, which keeps the comparison diverse with respect to model architectures, training data and instruction-tuning recipes. Table 2 lists the evaluated models with their context windows.

3.3. Prompt design and rationale

The 100 prompts in each language were prepared by hand rather than sampled from an existing benchmark, so that the experimental surface could be controlled along three dimensions: CWE coverage, task realism and absence of security framing. Each prompt was constructed by first identifying a security-sensitive task pattern that real developers are known to request from coding assistants, and then phrasing it as a functional request without any reference to security, hardening or input validation.

The prompt set is stratified across the CWE families that prior work has reported most frequently in AI-generated

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

code [7, 14, 22, 26, 31, 35], so that the design exercises the weaknesses that LLMs are empirically most likely to introduce rather than rare or contrived classes. The covered task categories include authentication and session management (e.g., login, token issuance, password storage), file and path handling (read/write of user-supplied paths, archive extraction), deserialization of untrusted data, SQL and OS command construction, cryptographic primitives (key generation, hashing, encryption mode selection), web input handling (request parsing, response rendering, cross-origin configuration) and resource allocation (file descriptors, memory buffers, regular expression matching against user input). Each of the four language-specific prompt sets covers the same task categories so that cross-language comparisons measure behavioural differences of the LLMs rather than differences in the prompt mix.

Each prompt asks the model to produce a complete, runnable solution and the exact wording is reused verbatim across all seven LLMs to remove prompt-phrasing as a source of variance. To avoid leading the model toward either secure or insecure code, every prompt was reviewed by both authors before inclusion against a small wording checklist: it must not mention vulnerabilities, security, sanitization, hardening, authentication strength, validation, encryption requirements, threat models or any CWE/CVE identifier; it must not contain wording that hints at common pitfalls (e.g., "be careful with"); and it must not over-specify the implementation in a way that constrains the model's choice of API or library. Any prompt that failed any of these checks was rephrased and re-reviewed before being added to the final set. The deliberate absence of any security-aware framing is intended to reflect the typical case in which a developer requests functionality without explicitly asking for hardened code, and it ensures that the resulting snippets characterize each LLM's baseline security behaviour rather than its response to security-specific prompting strategies, which we treat as out of scope for this study.

3.4. Two-stage vulnerability analysis

To reduce the false positive and false negative rates of any single tool, every snippet is analyzed in two stages. In the first stage, three SAST tools, Semgrep, Snyk and SonarCloud, are run independently on each snippet. All three tools support the four target languages and use largely disjoint rule sets, which broadens the coverage of detected weaknesses. Semgrep is used in its open-source CLI mode with the community and registry rule packs for each language. Snyk Code contributes its semantic flow-based analyses with a focus on authentication, injection and cryptographic misuse rules. SonarCloud adds the vulnerability and security hotspot rules for each target language, configured with the default quality profile. Findings from all three tools are deduplicated by location and CWE before being merged into a single per-snippet finding list.

In the second stage, every reported finding is reviewed manually by the authors. The manual stage serves two purposes. First, it discards false positives produced by overly aggressive SAST rules. Second, it captures logic-level weaknesses such as improper authorization checks or unsafe defaults that pattern-based tools miss. Two reviewers examined each reported finding independently, agreed on the validity of the finding and the appropriate CWE class, and a third reviewer arbitrated any disagreement. Each confirmed finding is mapped to a CWE category [43] and assigned a CVSS v3.1 severity (critical, high, medium or low) consistent with the CWE's typical exploitation profile.

3.5. Evaluation metrics

Four metrics are used to evaluate each LLM. The first two, Total Vulnerable Prompts (TVP) and Total Vulnerability Count (TVC), are the conventional metrics found in prior work. TVP is the number of generated snippets that contain at least one vulnerability and TVC is the total number of detected vulnerabilities across all snippets for the LLM. TVC additionally accounts for cases where multiple vulnerabilities co-occur in the same snippet, which TVP cannot capture. While simple and intuitive, TVP and TVC ignore variations in code size and severity. In our dataset Claude Sonnet 4.5 generates 54,788 lines of JavaScript for 100 prompts while Nova Pro generates only 7,621 lines; punishing Claude for verbosity by counting raw vulnerabilities would be misleading because the same prompt set has produced two snippets with very different surface areas and therefore very different opportunities for the SAST tools and reviewers to find weaknesses.

To address output-size variance we define

$$V/LoC = \frac{\sum_{i=1}^{n} V_i}{LoC} \tag{1}$$

- V/LoC: Vulnerabilities per Line of Code
- V_i: Frequency of the ith vulnerability in the generated code
- n: Total number of distinct CWE categories observed
- LoC: Number of lines of code generated for the LLM

V/LoC normalizes the raw vulnerability count to the size of the code surface that produced it and is the natural denominator when the cost of human review scales with code volume. To additionally account for severity differences we define

$$WSR/LoC = \frac{\sum_{i=1}^{n} (CVSS_i \cdot V_i)}{LoC} \tag{2}$$

- WSR/LoC: Weighted Security Risk per Line of Code
- $CVSS_i$: Common Vulnerability Scoring System (CVSS) score of the ith vulnerability
- V_i: Frequency of the ith vulnerability in the generated code
- LoC: Number of lines of code generated for the LLM

By weighting each vulnerability by its CVSS severity, WSR/LoC treats a few critical findings as more harmful than

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

Table 3
CWE frequency in AI-generated JavaScript code.

CWE	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
CWE-200	28	33	19	14	21	44	23	182
CWE-319	44	6	5	4	35	19	6	119
CWE-79	6	25	13	16	20	7	11	98
CWE-770	7	8	10	0	8	26	8	67
CWE-1287	9	13	1	0	6	7	0	36
CWE-23	4	2	3	2	3	14	4	32
CWE-89	2	3	4	5	14	2	2	32
CWE-942	0	0	8	0	0	0	18	26
CWE-1333	0	2	10	0	1	1	3	17
CWE-327	1	1	3	0	2	4	3	14
CWE-117	0	2	1	5	3	2	0	13
CWE-918	1	1	1	1	1	5	3	13
CWE-94	0	0	0	0	1	8	0	9
Others	0	2	7	1	12	9	6	37
Total	**102**	**98**	**85**	**48**	**127**	**148**	**87**	**695**

many low-severity findings, which better reflects the security risk an AI-assisted developer faces in practice. Equation 1 is mathematically equivalent to TVC/LoC, since summing V_i across all CWEs gives the total vulnerability count.

4. Results

A two-stage analysis of the 2,800 generated snippets identifies 1,703 vulnerabilities in 936 snippets (33.4%). The vulnerabilities are unevenly distributed across the four languages: 695 in JavaScript, 398 in Python, 348 in Java and 262 in C++. The rest of this section reports per-language CWE distributions, severity profiles and metric values, and then consolidates the findings in a cross-language comparison.

4.1. JavaScript

JavaScript exhibits the highest vulnerability density. Of the 700 generated snippets, 346 (49.4%) contain at least one vulnerability and 695 vulnerabilities are detected across 25 distinct CWEs. Mistral Large 2 produces the highest TVC (148), followed by Llama 3.1 405B (127) and GPT-5 (102); Gemini 3 Flash produces the fewest (48). Code volume varies considerably across LLMs: Claude Sonnet 4.5 generates the largest amount of JavaScript code at 54,788 lines, followed by Gemini 3 Flash (15,274) and Mistral Large 2 (14,429), while Nova Pro generates only 7,621 lines and GPT-5 generates 9,758. Despite producing a moderate amount of code, Mistral Large 2 and Llama 3.1 405B record the highest vulnerability counts, which suggests that the nature of the generated code matters more for security risk than its sheer quantity. The full CWE breakdown is shown in Table 3.

The most common CWE in AI-generated JavaScript code is CWE-200 (Information Exposure), which appears

Table 4
Severity distribution of vulnerabilities in AI-generated JavaScript code.

Severity	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
Critical	2	3	4	5	14	2	2	32
High	7	4	13	3	42	38	15	122
Medium	93	73	57	35	55	98	52	463
Low	0	18	11	5	16	10	18	78

182 times and accounts for 26.2% of all reported vulnerabilities. CWE-319 (Cleartext Transmission of Sensitive Information) follows with 119 occurrences (17.1%) and indicates a persistent tendency to send sensitive data over unprotected channels. CWE-79 (Cross-Site Scripting) appears 98 times (14.1%) and highlights the risk of insecure client-side scripting. CWE-770 (Allocation of Resources Without Limits) appears 67 times (9.6%) and CWE-1287 (Improper Validation of Specified Type of Input) appears 36 times (5.2%). Together the five most common CWEs account for 72.2% of detected vulnerabilities. The per-LLM distribution of these CWEs is markedly uneven. Mistral Large 2 concentrates strongly on CWE-200 (44 instances), Llama 3.1 405B produces the largest count of CWE-89 (SQL Injection) at 14 instances, Claude Sonnet 4.5 leads CWE-79 with 25 occurrences and DeepSeek R1 70B distributes its weaknesses more evenly while still leading CWE-1333 (ReDoS) with 10 instances. Nova Pro shows an unusually high concentration of CWE-942 (Overly Permissive Cross-Domain Policy) with 18 cases, which suggests a recurring issue with default cross-origin configurations in its JavaScript output. These regularities are most likely caused by differences in model architectures, training corpora and instruction-tuning policies.

The severity profile in Table 4 and Figure 3 is medium-dominated (66.6%), followed by high (17.6%), low (11.2%) and critical (4.6%). The 32 critical findings are concentrated in Llama 3.1 405B (14 out of 32), while the high severity findings are distributed more evenly with Llama (42) and Mistral Large 2 (38) contributing the most. The low severity findings are dominated by Nova Pro (18) and Claude Sonnet 4.5 (18). All seven models therefore produce vulnerabilities, but the severity distribution varies markedly between them and mirrors the variability already seen in the CWE distribution.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

Table 5
JavaScript: per-LLM metrics. Bold marks the best value in each row.

Metric	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro
TVP	73	56	38	**21**	60	60	38
TVC	102	98	85	**48**	127	148	87
V/LoC	0.0105	**0.0018**	0.0061	0.0031	0.0116	0.0103	0.0114
WSR/LoC	0.0605	**0.0091**	0.0350	0.0180	0.0754	0.0624	0.0615

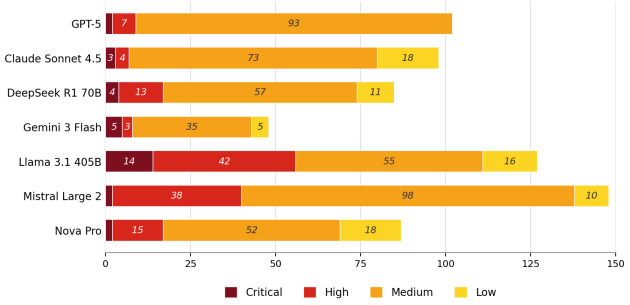


Figure 3: Severity of vulnerabilities in JavaScript code.

Code volume varies substantially across the seven LLMs, and the metric rankings therefore differ from the raw counts. Although Mistral Large 2 records the highest TVC, its normalized V/LoC of 0.0103 is comparable to GPT-5 (0.0105) and lower than Llama 3.1 405B (0.0116) and Nova Pro (0.0114). Claude Sonnet 4.5 emerges as the most secure model under V/LoC (0.0018) and WSR/LoC (0.0091), and Llama 3.1 405B is the worst on both metrics (0.0116 and 0.0754 respectively). The contrast between Claude Sonnet 4.5's raw vulnerability count and its normalized scores illustrates how V/LoC and WSR/LoC correct for verbosity bias: a model that writes substantially more code per prompt should not be penalised for producing more raw findings if those findings are spread across a much larger and lower-density surface. Table 5 summarises the per-LLM scores.

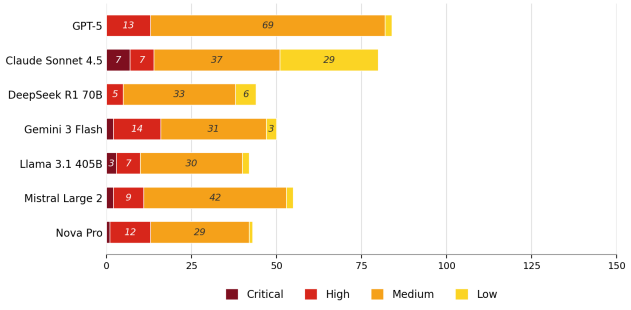
4.2. Python

Python contains 398 vulnerabilities across 25 CWEs in 238 vulnerable snippets (34.0%). GPT-5 leads both TVP (56) and TVC (84), and Claude Sonnet 4.5 follows closely with 80 vulnerabilities. Code volume again varies by almost an order of magnitude: Claude Sonnet 4.5 generates 58,131 lines, Gemini 3 Flash 18,072, DeepSeek R1 70B 14,370, GPT-5 11,217, Mistral Large 2 8,525, Llama 3.1 405B 7,717 and Nova Pro 6,821. The small line counts produced by Llama and Nova Pro partly explain their relatively low TVC values, since a smaller code surface generally exposes fewer flaws. The dominant CWE is CWE-23 (Relative Path Traversal) at 91 cases (22.9%), followed by CWE-352 (Cross-Site Request Forgery) at 75 cases (18.8%), CWE-89 (SQL Injection) at 42, CWE-78 (OS Command Injection) at 27 and CWE-117 (Improper Output Neutralization for Logs) at 27. Together the five most common CWEs account for 65.8% of detected weaknesses. The full breakdown is shown in Table 6.

Table 6
CWE frequency in AI-generated Python code.

CWE	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
CWE-23	35	8	9	11	4	14	10	91
CWE-352	1	12	7	11	13	16	15	75
CWE-89	12	11	3	7	6	2	1	42
CWE-78	8	3	4	2	4	3	3	27
CWE-117	0	18	2	3	1	2	1	27
CWE-918	4	1	6	3	3	6	1	24
CWE-942	2	11	4	0	1	0	0	18
CWE-1236	15	0	0	0	0	0	0	15
CWE-22	2	5	4	1	0	0	1	13
CWE-94	0	0	1	4	4	0	3	12
CWE-79	0	3	2	1	1	1	1	9
Others	5	8	2	7	5	11	7	45
Total	**84**	**80**	**44**	**50**	**42**	**55**	**43**	**398**

The per-LLM concentrations on the Python side are also striking. GPT-5 produces 35 of the 91 CWE-23 cases, which indicates a recurring tendency to construct file system paths without canonicalisation, and it is the only LLM that produces CWE-1236 (Improper Neutralization of Formula Elements in a CSV File) with 15 cases, all within prompts that ask the model to serialise tabular data. Claude Sonnet 4.5 produces 18 of the 27 CWE-117 cases (Improper Output Neutralization for Logs) and 11 of the 18 CWE-942 cases, which together suggest that its Python output tends to under-sanitise non-functional channels such as logs and HTTP response headers. Mistral Large 2 contributes the largest share of CWE-352 (16 cases), Llama 3.1 405B follows with 13 cases, and Nova Pro adds 15, which indicates that CSRF mitigation defaults in web-facing Python code are a cross-model weakness. The severity profile is again dominated by medium findings (68.1%), with high (16.8%), low (11.3%) and critical (3.8%) following, as shown in Table 7 and Figure 4. Critical vulnerabilities are concentrated in Claude Sonnet 4.5 (7 of 15), while the high severity category is led by Gemini 3 Flash with 14 cases.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

Table 7
Severity distribution of vulnerabilities in AI-generated Python code.

Severity	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
Critical	0	7	0	2	3	2	1	15
High	13	7	5	14	7	9	12	67
Medium	69	37	33	31	30	42	29	271
Low	2	29	6	3	2	2	1	45

Table 8
Python: per-LLM metrics. Bold marks the best value in each row.

Metric	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro
TVP	56	39	32	29	25	33	**24**
TVC	84	80	44	50	**42**	55	43
V/LoC	0.0075	**0.0014**	0.0031	0.0028	0.0054	0.0065	0.0063
WSR/LoC	0.0440	**0.0067**	0.0164	0.0174	0.0333	0.0388	0.0400

Figure 4: Severity of vulnerabilities in Python code.

Claude Sonnet 4.5 again has the lowest V/LoC (0.0014) and WSR/LoC (0.0067); GPT-5 produces the highest WSR/LoC at 0.0440, in spite of writing only 11,217 lines, because its CWE-23 and CWE-1236 concentrations sit in code that the prompts treat as the main control surface. The Python data also produces a clean example of the metric divergence: under V/LoC, Gemini 3 Flash (0.0028) is slightly better than DeepSeek R1 70B (0.0031), but under WSR/LoC the order reverses (0.0174 vs 0.0164) because DeepSeek's findings skew less severe. Per-LLM metrics are reported in Table 8.

4.3. C++

C++ produces the fewest vulnerabilities in absolute terms (262 across 20 CWEs) but the smaller count reflects a smaller code volume and a narrower CWE spectrum rather than safer behaviour. The number of generated lines varies sharply across LLMs: Claude Sonnet 4.5 produces 49,835 lines, Gemini 3 Flash 25,519, DeepSeek R1 70B 12,352, Llama 3.1 405B 8,723, GPT-5 8,447, Mistral Large 2 8,004 and Nova Pro only 7,078. Despite this volume, Claude Sonnet 4.5 dominates the absolute counts with 82 vulnerabilities, but GPT-5 reports only 9 (the smallest count observed for any LLM in any language). CWE-23 (Relative Path Traversal) dominates with 105 occurrences (40.1%), followed by CWE-190 (Integer Overflow) with 57 (21.8%), CWE-326 (Inadequate Encryption Strength) with 27 (10.3%), CWE-125 (Out-of-bounds Read) with 14 (5.3%) and CWE-78 with 10 (3.8%). Together the top five CWEs account for 81.3% of detected weaknesses, which is the highest concentration observed in the study and reflects both the narrower attack surface of the C++ prompts and the recurring memory- and arithmetic-safety patterns that the language exposes. The full breakdown is shown in Table 9.

Table 9
CWE frequency in AI-generated C++ code.

CWE	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
CWE-23	1	30	20	15	16	14	9	105
CWE-190	1	34	10	6	1	3	2	57
CWE-326	0	0	4	4	6	5	8	27
CWE-125	4	4	2	0	2	2	0	14
CWE-78	1	1	2	1	2	3	0	10
CWE-122	0	5	0	0	2	0	0	7
CWE-918	0	2	1	2	1	1	0	7
CWE-120	0	2	0	1	1	0	2	6
CWE-476	0	0	1	0	2	3	0	6
CWE-922	0	0	2	1	1	0	1	5
Others	2	4	2	1	1	2	6	18
Total	**9**	**82**	**44**	**31**	**35**	**33**	**28**	**262**

Table 10
Severity distribution of vulnerabilities in AI-generated C++ code.

Severity	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
Critical	0	1	0	0	1	0	0	2
High	6	6	5	1	4	5	2	29
Medium	3	74	38	28	30	27	23	223
Low	0	1	1	2	0	1	3	8

The per-LLM C++ profile is the most uneven of the four languages. Claude Sonnet 4.5 alone produces 34 of the 57 CWE-190 cases and 30 of the 105 CWE-23 cases, which indicates a tendency to generate arithmetic that the language's integer rules do not guard against and file-handling code that does not canonicalise paths. DeepSeek R1 70B, Llama 3.1 405B, Gemini 3 Flash and Mistral Large 2 each contribute 14–20 CWE-23 cases, confirming that CWE-23 is a cross-model weakness in C++. Nova Pro is the only LLM that produces CWE-401 (memory leak) cases and contributes 2 of the 3 CWE-787 (Out-of-bounds Write) findings, which together point to recurring memory-management issues. GPT-5's CWE-125 (Out-of-bounds Read) count of 4 is notable in spite of its very low overall TVC. The severity profile is heavily medium-dominated (85.1%), with critical and high together accounting for only 11.8%, as shown in Table 10 and Figure 5.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

Table 11
C++: per-LLM metrics. Bold marks the best value in each row.

Metric	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro
TVP	**9**	44	26	23	24	22	21
TVC	**9**	82	44	31	35	33	28
V/LoC	**0.0011**	0.0016	0.0036	0.0012	0.0040	0.0041	0.0040
WSR/LoC	0.0080	0.0094	0.0200	**0.0067**	0.0235	0.0246	0.0221

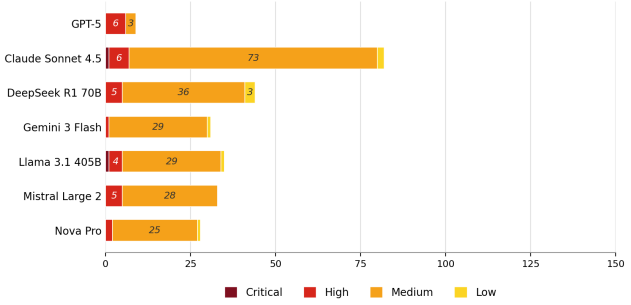


Figure 5: Severity of vulnerabilities in C++ code.

Under V/LoC GPT-5 is the most secure (0.0011) and Mistral Large 2 the least secure (0.0041); under WSR/LoC Gemini 3 Flash takes the top position (0.0067), showing that V/LoC and WSR/LoC can produce different rankings for the same model. The Mistral Large 2 case is informative: it produces fewer absolute vulnerabilities than Claude Sonnet 4.5 (33 vs 82), but its V/LoC is higher (0.0041 vs 0.0016) because its code volume is much smaller (8,004 vs 49,835 lines), and its WSR/LoC of 0.0246 is the highest in the table, which means that the relative severity-weighted risk of its C++ output exceeds Claude Sonnet 4.5's in spite of the much lower TVC. Per-LLM metrics are reported in Table 11.

4.4. Java

Java contains 348 vulnerabilities across 22 CWEs in 183 vulnerable snippets (26.1%). Code volume in Java is consistently large across all LLMs except Nova Pro, ranging from Claude Sonnet 4.5 at 57,568 lines through Gemini 3 Flash at 38,721 and GPT-5 at 35,618, down to Mistral Large 2 at 14,773, Llama 3.1 405B at 11,899 and Nova Pro at 10,252. The relatively long average snippet length in Java reflects the language's verbose syntax framework and it shapes the V/LoC and WSR/LoC values that follow. Claude Sonnet 4.5 produces the highest TVC (107) and TVP (46), while Llama 3.1 405B produces the fewest vulnerabilities overall (24) and the lowest TVP (19). The top CWEs are CWE-23 (63, 18.1%), CWE-319 (49, 14.1%), CWE-352 (32, 9.2%), CWE-502 (Deserialization of Untrusted Data) (26, 7.5%) and CWE-942 (22, 6.3%). The full breakdown is shown in Table 12.

Table 12
CWE frequency in AI-generated Java code.

CWE	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
CWE-23	15	13	11	7	3	6	8	63
CWE-319	5	12	8	6	7	3	8	49
CWE-352	1	13	3	4	3	7	1	32
CWE-502	0	6	4	12	0	2	2	26
CWE-942	1	21	0	0	0	0	0	22
CWE-79	4	1	2	4	1	5	3	20
CWE-918	2	6	4	3	0	0	2	17
CWE-78	4	2	3	2	0	2	2	15
CWE-89	1	6	1	1	0	1	3	13
CWE-326	1	4	1	2	1	3	0	12
CWE-611	2	1	2	2	1	1	3	12
CWE-470	3	0	0	2	2	1	3	11
Others	4	22	21	8	6	9	11	56
Total	**33**	**107**	**60**	**38**	**24**	**40**	**46**	**348**

Table 13
Severity distribution of vulnerabilities in AI-generated Java code.

Severity	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro	Total
Critical	1	0	0	1	0	0	1	3
High	12	28	15	18	4	12	20	109
Medium	29	57	28	24	17	20	23	198
Low	1	22	0	4	3	7	1	38

Per-LLM CWE concentrations in Java are even more pronounced than in the other languages. Claude Sonnet 4.5 alone produces 21 of the 22 CWE-942 cases and 13 of the 32 CWE-352 cases, which together indicate a recurring tendency to default to permissive cross-domain policies and to omit CSRF mitigations in web-facing Java code. Gemini 3 Flash produces 12 of the 26 CWE-502 cases, which is consistent with a recurring usage of unsafe deserialization defaults. GPT-5 produces 15 of the 63 CWE-23 cases, which mirrors its Python behaviour and suggests that GPT-5's default file-handling idiom is consistently uncanonicalised across both languages. Java differs noticeably from the other three languages in its severity profile: high severity findings account for 31.3% of all detections, which is nearly double the high-severity ratio observed in JavaScript and Python and approximately three times that of C++. Table 13 and Figure 6 present the severity breakdown.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

Table 14
Java: per-LLM metrics. Bold marks the best value in each row.

Metric	GPT-5	Claude 4.5	DeepSeek	Gemini 3F	Llama 405B	Mistral L2	Nova Pro
TVP	22	46	20	21	**19**	24	24
TVC	33	107	60	38	**24**	40	46
V/LoC	**0.0012**	0.0019	0.0026	**0.0012**	0.0020	0.0026	0.0044
WSR/LoC	**0.0077**	0.0103	0.0168	0.0078	0.0112	0.0153	0.0300

Table 15
Pooled WSR/LoC across the four languages.

LLM	JS	Py	C++	Java	Pooled
GPT-5	0.0605	0.0440	0.0080	**0.0077**	0.0219
Claude Sonnet 4.5	**0.0091**	**0.0067**	0.0094	0.0103	**0.0088**
DeepSeek R1 70B	0.0350	0.0164	0.0200	0.0168	0.0218
Gemini 3 Flash	0.0180	0.0174	**0.0067**	0.0078	0.0109
Llama 3.1 405B	0.0754	0.0333	0.0235	0.0112	0.0362
Mistral Large 2	0.0624	0.0388	0.0246	0.0153	0.0362
Nova Pro	0.0615	0.0400	0.0221	0.0300	0.0380

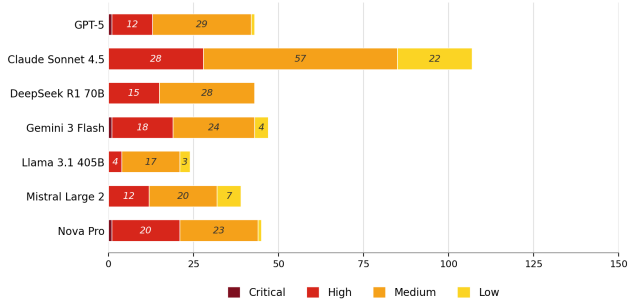


Figure 6: Severity of vulnerabilities in Java code.

As a result the gap between V/LoC and WSR/LoC rankings is the largest in Java. GPT-5 has the lowest V/LoC (0.0012) and the lowest WSR/LoC (0.0077), tied closely with Gemini 3 Flash; Nova Pro has the highest WSR/LoC (0.0300) and the highest V/LoC (0.0044). DeepSeek R1 70B has a higher WSR/LoC score (0.0168) than Mistral Large 2 (0.0153) even though they share a similar V/LoC of 0.0026, because the relative severity of DeepSeek R1 70B's findings is higher. Per-LLM metrics are reported in Table 14.

4.5. Cross-language analysis

Pooling the per-language results exposes patterns that no single language reveals. The proportion of vulnerable prompts varies considerably between the four programming languages. JavaScript exhibits the highest vulnerability density at 49.4%, Python follows at 34.0%, and Java and C++ produce vulnerable code in 26.1% and 24.1% of cases respectively. The same ordering is observed in the absolute vulnerability counts (695, 398, 348 and 262). JavaScript therefore emerges as the language in which AI-generated code is most prone to security weaknesses, while C++ contains the fewest detected vulnerabilities in absolute terms, even though the lower count is partly caused by the smaller code volume produced for C++ and by the narrower CWE spectrum, with 20 distinct CWEs identified in C++ compared with 25–26 distinct CWEs in the other three languages. The severity distribution also shifts noticeably between languages: C++ is dominated by medium severity findings (85.1%) while Java carries the largest high-severity ratio (31.3%), so approximately one in every three vulnerabilities in AI-generated Java code is of high severity, compared to roughly one in five in JavaScript and Python and one in nine in C++.

Table 15 reports the pooled WSR/LoC, computed by summing the severity-weighted vulnerability scores across all four languages and dividing by the total generated lines of code for the LLM. Pooled V/LoC follows the same definition with V_i replacing $CVSS_i \cdot V_i$.

Three observations stand out. First, LLM rankings reverse across languages. GPT-5 ranges from the worst on JavaScript (0.0605) to the best on Java (0.0077), a nearly eightfold swing. Llama 3.1 405B is the worst in three of four languages but the second best in C++ under V/LoC. Claude Sonnet 4.5 leads the pooled metrics with WSR/LoC 0.0088 despite producing the highest TVC in C++ and Java; its consistently large code generation volume rather than fewer findings is what places it at the top. The variation of Claude Sonnet 4.5's V/LoC across the four languages is only 0.0005 (0.0014–0.0019), which is the most stable cross-language behaviour observed in the study and an important characteristic for teams that work in multiple languages with a single model.

Second, each LLM concentrates on a different subset of CWEs and the dominant CWE for a given model varies by language. Mistral Large 2 leads CWE-200 in JavaScript (44 instances) but CWE-352 in Python (16). Llama 3.1 405B accounts for the highest CWE-89 count in JavaScript (14) but produces almost no Java vulnerabilities. Claude Sonnet 4.5 dominates CWE-190 in C++ (34), CWE-942 in Java (21) and CWE-117 in Python (18). GPT-5 is the only LLM that produces CWE-1236 CSV formula injection in Python (15 cases) and leads CWE-319 in JavaScript (44). DeepSeek R1 70B is the largest contributor to CWE-1333 ReDoS in JavaScript (10), and Nova Pro produces all observed CWE-401 memory leak cases in C++ and the largest CWE-942 share in JavaScript (18). These model-specific concentrations effectively act as priors that a downstream review pipeline can encode as additional checks when a particular LLM is used in a particular language.

Third, V/LoC and WSR/LoC disagree in non-trivial ways. For Python, Gemini 3 Flash has a better V/LoC than DeepSeek R1 70B (0.0028 vs 0.0031) but a worse WSR/LoC (0.0174 vs 0.0164), because Gemini's findings skew slightly more severe. In Java, Nova Pro has the highest V/LoC and the highest WSR/LoC, and the relative gap is larger under WSR/LoC, indicating both higher density

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

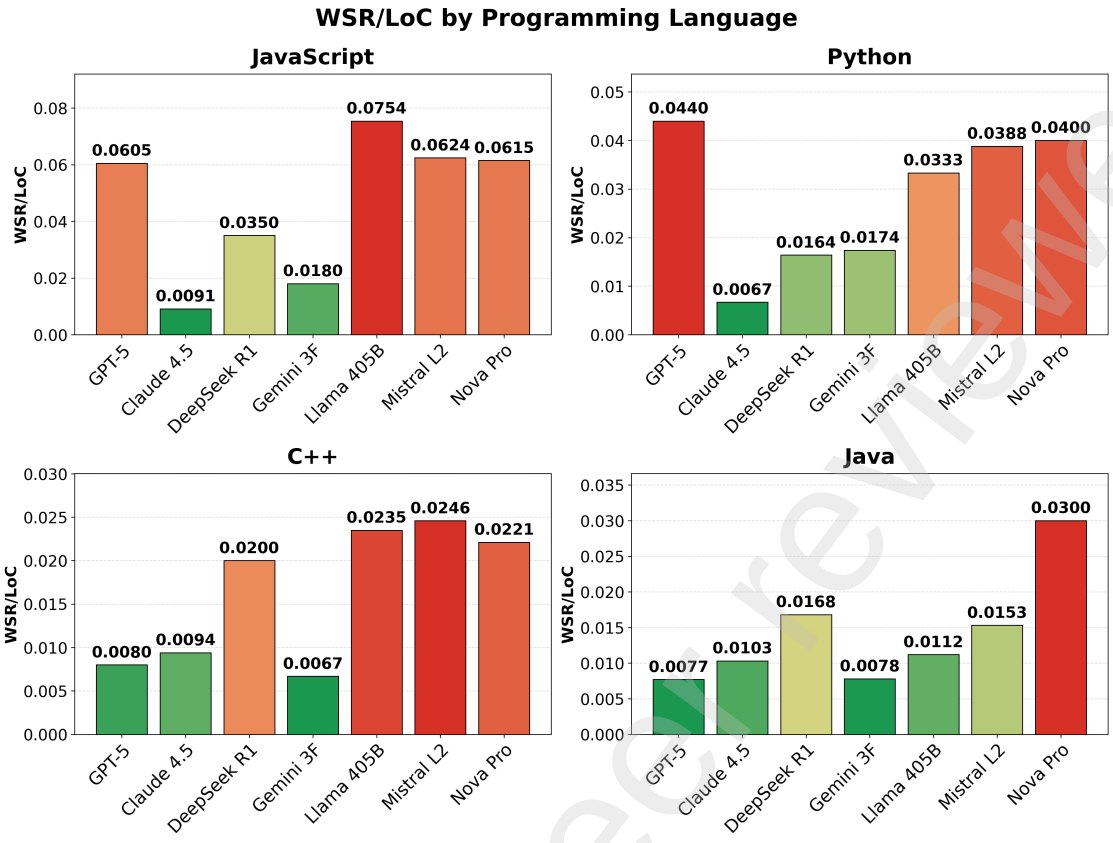


Figure 7: WSR/LoC comparison across LLMs by programming language. Lower is better.

and higher severity. The pooled column also illustrates the case clearly: Llama 3.1 405B and Mistral Large 2 produce identical pooled WSR/LoC (0.0362) even though their per-language V/LoC values differ, because their cumulative severity weight balances out. A comparison of LLM WSR/LoC performance across the four programming languages is shown in Figure 7.

4.6. Discussion

The results have several practical implications for AI-assisted software development. The most striking finding is that no LLM in the evaluation is consistently the safest across all four languages. Claude Sonnet 4.5 has the strongest pooled metrics but is not the best in C++ or Java. GPT-5 is exceptional in C++ and Java but produces the second largest WSR/LoC in JavaScript. The implication is that the model selected for a given task should be informed by the target language and not by a generic "best model" belief.

The same data tell different stories depending on the metric. Counting raw vulnerabilities ranks Claude Sonnet 4.5 as the worst Java model with 107 findings, but normalizing by output size and severity ranks it among the safest. Conversely Llama 3.1 405B looks competitive on raw counts in some languages while its severity-weighted score is the worst in three of four. V/LoC and WSR/LoC are inexpensive to compute from the same SAST output and consistently reveal the gap between raw activity and actual risk.

The dominance of CWE-23 path traversal in C++ (40.1%), Python (22.9%) and Java (18.1%) but not in JavaScript suggests that LLMs default to writing file-handling code without input canonicalization in any language that exposes a straightforward file system API. JavaScript's high contribution from CWE-200 and CWE-319 reflects a different pattern: the models tend to leak sensitive information or transmit it without encryption in web-facing code. The severity skew in Java (31.3% high severity) likely reflects the prevalence of enterprise patterns such as unsafe deserialization and overly permissive cross-domain configuration. These regularities argue for language-aware secure coding guidelines for LLM-generated output rather than generic prompt hardening.

Because each LLM concentrates on a distinct CWE subset, a review pipeline can be tuned to its model. For example, code generated by Mistral Large 2 in JavaScript would benefit from extra scanning for CWE-200 and CWE-770; Claude Sonnet 4.5 output in Java should be scrutinized for permissive cross-domain configuration; and GPT-5 in Python warrants CSV-output review for CWE-1236. Practitioners can use the per-LLM CWE distributions as model-specific risk priors when configuring SAST rule sets or human review checklists.

Four kinds of threats most directly affect this study. Regarding construct validity, V/LoC and WSR/LoC depend on the unit of code volume and on the CVSS base score. We

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

use LoC computed after removing blank lines and comment-only lines, and CVSS v3.1 base scores from the National Vulnerability Database where available and the typical CWE exploitation profile otherwise. Different choices of normalizer (for example function points or AST node count) and severity weighting would shift the absolute values, but the ranking trends we report are robust to small changes in these definitions. Regarding internal validity, a single SAST tool would produce both false positives and false negatives; we mitigate this by combining three independent tools and reconciling their findings with manual review by two reviewers and a third arbiter. External validity is bounded by the fixed prompt set of 100 tasks per language and the specific LLMs and languages chosen, which represent a slice of current practice rather than the entire ecosystem. Reproducibility is bounded by the fact that LLM versions evolve quickly; we mitigate this by recording the model identifier, the API version and the decoding parameters for every snippet, and we release the prompt list, the generated snippets and the labelled finding set as supplementary material.

The study is also bounded by three additional constraints. First, the evaluation rests on SAST plus manual review and does not exercise dynamic or fuzz testing, which could expose runtime-only weaknesses such as race conditions or deserialization gadgets that only trigger with specific input. Second, the LoC-based normalization does not capture the security relevance of every line; a single critical operation can dominate the risk of a snippet that contains many benign helper lines, and a per-snippet severity-weighted normalisation would complement the per-line view used here. Third, the CVSS severity used in WSR/LoC is a base score and does not include the environmental or temporal modifiers that a real deployment would apply, so absolute WSR/LoC values should be read as comparative scores rather than as deployment-ready risk estimates.

5. Conclusion and Future Work

We introduced a controlled, multi-model and multi-language security evaluation of AI-generated code that covers seven contemporary LLMs and four widely used programming languages across 2,800 generated snippets. Under pooled WSR/LoC, the most secure LLM in our evaluation was 4.3× safer than the least secure, and the same model can swing by up to 7.9× depending on the language used. These results show that no single model dominates the others in all four languages. The analysis identified 1,703 vulnerabilities in 936 snippets (33.4%), unevenly distributed across the four languages, and concentrated on a small set of CWEs that differs by language. We introduced two new evaluation metrics, V/LoC and WSR/LoC, which normalize vulnerability counts by output size and severity, respectively. Additionally, we empirically showed that they expose risks that raw counts hide and they reveal different LLM rankings depending on the language. JavaScript exhibits the highest vulnerability density at 49.4% while C++ produces the fewest absolute findings. Java carries the largest high-severity share at 31.3%. These results motivate language-aware and model-aware security review in AI-assisted software development.

Future work can expand this research in several directions. The prompt set may be extended and additional programming languages may be added to improve external validity. Dynamic analysis, fuzzing, and end-to-end runtime testing can complement the static plus manual pipeline used in this research.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by Odine and OdineLabs as part of the ImAgSLab Project, under the 1773 ITU TTO Project No: 202500303.

Acknowledgments

The authors thank Odine and OdineLabs for their support of the ImAgSLab Project.

Data availability

The full prompt list, the generated code snippets and the labelled vulnerability findings are openly available at https://github.com/AydinDeniz/vloc-wsrloc.

CRediT authorship contribution statement

Deniz Aydın: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Şerif Bahtiyar:** Conceptualization, Methodology, Validation, Writing – review & editing, Supervision, Project administration, Funding acquisition.

References

[1] Aldosari, S.S., Aldawsari, L.S., 2026. Securing LLM code generation: Leveraging prompt engineering to mitigate vulnerabilities across models and languages. Science of Computer Programming 251, 103446. doi:10.1016/j.scico.2026.103446.

[2] Almanasra, S., Suwais, K., 2025. Analysis of ChatGPT-generated codes across multiple programming languages. IEEE Access 13, 23580–23596. doi:10.1109/ACCESS.2025.3538050.

[3] Amazon, 2024. Nova pro. URL: https://aws.amazon.com/ai/generative-ai/nova/.

[4] Ambati, S.H., Ridley, N., Branca, E., Stakhanova, N., 2024. Navigating (in)Security of AI-generated code, in: 2024 IEEE International Conference on Cyber Security and Resilience (CSR), IEEE, London, United Kingdom. pp. 1–8. doi:10.1109/CSR61664.2024.10679468.

[5] Anthropic, 2025. Claude Sonnet 4.5. URL: https://www.anthropic.com/news/claude-sonnet-4-5.

[6] Asare, O., Nagappan, M., Asokan, N., 2023. Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code? Empirical Software Engineering 28. doi:10.1007/s10664-023-10380-1.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

[7] Aydın, D., Bahtiyar, Ş., 2025. Security vulnerabilities in AI-generated JavaScript: A comparative study of large language models, in: 2025 IEEE International Conference on Cyber Security and Resilience (CSR), IEEE, Chania, Crete, Greece. pp. 200–205. doi:10.1109/CSR64739.2025.11130176.

[8] Belozerov, V., Barclay, P.J., Sami, A., 2026. Secure coding with AI – from detection to repair. Empirical Software Engineering doi:10.1007/s10664-026-10812-8.

[9] Bruni, M., Gabrielli, F., Ghafari, M., Kropp, M., 2025. Benchmarking prompt engineering techniques for secure code generation with GPT models. URL: https://arxiv.org/abs/2502.06039. arXiv preprint arXiv:2502.06039.

[10] Daigle, K., 2024. Survey: The AI wave continues to grow on software development teams. URL: https://github.blog/news-insights/research/survey-ai-wave-grows/.

[11] DeepSeek AI, 2024. DeepSeek R1 70B: A large-scale language model. URL: https://www.deepseek.com.

[12] Dubniczky, R.A., Horvát, K.Z., Bisztray, T., Ferrag, M.A., Cordeiro, L.C., Tihanyi, N., 2025. CASTLE: Benchmarking dataset for static code analyzers and LLMs towards CWE detection, in: Proceedings of the 19th International Symposium on Theoretical Aspects of Software Engineering (TASE 2025), Springer.

[13] Firouzi, E., Ghafari, M., 2026. Persistent human feedback, LLMs, and static analyzers for secure code generation and vulnerability detection. URL: https://arxiv.org/abs/2602.05868. arXiv preprint arXiv:2602.05868.

[14] Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., Yu, J., Chen, J., 2025. Security weaknesses of Copilot-generated code in GitHub projects: An empirical study. ACM Transactions on Software Engineering and Methodology doi:10.1145/3716848.

[15] Gasiba, T.E., Iosif, A.C., Kessba, I., Amburi, S., Lechner, U., Pinto-Albuquerque, M., 2024. May the source be with you: On ChatGPT, cybersecurity, and secure coding. Information 15, 572. doi:10.3390/info15090572.

[16] Google, 2025. Gemini 3 Flash. URL: https://deepmind.google/technologies/gemini/.

[17] Hajipour, H., Hassler, K., Holz, T., Schönherr, L., Fritz, M., 2024. CodeLMSec benchmark: Systematically evaluating and finding security vulnerabilities in black-box code language models, in: 2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), IEEE, Toronto, ON, Canada. pp. 684–709. doi:10.1109/SaTML59370.2024.00040.

[18] Hamer, S., d'Amorim, M., Williams, L., 2024. Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers, in: 2024 IEEE Security and Privacy Workshops (SPW), IEEE, San Francisco, CA, USA. pp. 87–94. doi:10.1109/SPW63631.2024.00014.

[19] Huynh, L., Zhang, Y., Jayasundera, D., Jeon, W., Kim, H., Bi, T., Hong, J.B., 2025. Detecting code vulnerabilities using LLMs, in: 2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, Naples, Italy. pp. 401–414. doi:10.1109/DSN64029.2025.00047.

[20] Khare, A., Dutta, S., Li, Z., Solko-Breslin, A., Alur, R., Naik, M., 2025. Understanding the effectiveness of large language models in detecting security vulnerabilities, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, Napoli, Italy. pp. 103–114. doi:10.1109/ICST62969.2025.10988968.

[21] Kharma, M.F., Choi, S., Alkhanafseh, M., Mohaisen, D., 2026. Security and quality in LLM-generated code: A multi-language, multi-model analysis. IEEE Transactions on Dependable and Secure Computing URL: https://doi.ieeecomputersociety.org/10.1109/TDSC.2026.3672745, doi:10.1109/TDSC.2026.3672745. early Access.

[22] Khoury, R., Avila, A.R., Brunelle, J., Camara, B.M., 2023. How secure is code generated by ChatGPT?, in: 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, Honolulu, HI, USA. pp. 2445–2451. doi:10.1109/SMC53992.2023.10394237.

[23] Lekssays, A., Mouhcine, H., Tran, K., Yu, T., Khalil, I., 2025. LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models, in: Proceedings of the 34th USENIX Security Symposium, USENIX Association.

[24] Li, F., Jiang, J., Chen, D., Xiong, Y., 2026. LLM-based vulnerability detection at project scale: An empirical study. URL: https://arxiv.org/abs/2601.19239. arXiv preprint arXiv:2601.19239.

[25] Li, X., Ding, J., Peng, C., Zhao, B., Gao, X., Gao, H., Gu, X., 2025. SafeGenBench: A benchmark framework for security vulnerability detection in LLM-generated code. URL: https://arxiv.org/abs/2506.05692. arXiv preprint arXiv:2506.05692.

[26] Majdinasab, V., Bishop, M.J., Rasheed, S., Moradidakhel, A., Tahir, A., Khomh, F., 2024. Assessing the security of GitHub Copilot's generated code – a targeted replication study, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, Rovaniemi, Finland. pp. 435–444. doi:10.1109/SANER60148.2024.00051.

[27] Meta AI, 2024. Llama v3.1 405B. URL: https://ai.meta.com/llama.

[28] Mistral AI, 2024. Mistral large 2: High-performance language model. URL: https://mistral.ai.

[29] Negri-Ribalta, C., Geraud-Stewart, R., Sergeeva, A., Lenzini, G., 2024. A systematic literature review on the impact of AI models on the security of code generation. Frontiers in Big Data 7, 1386720. doi:10.3389/fdata.2024.1386720.

[30] OpenAI, 2025. GPT-5. URL: https://openai.com/index/introducing-gpt-5/.

[31] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R., 2022. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions, in: 2022 IEEE Symposium on Security and Privacy (SP), IEEE, San Francisco, CA, USA. pp. 754–768. doi:10.1109/SP46214.2022.9833571.

[32] Perry, N., Srivastava, M., Kumar, D., Boneh, D., 2023. Do users write more insecure code with AI assistants?, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23), ACM, Copenhagen, Denmark. pp. 2785–2799. doi:10.1145/3576915.3623157.

[33] Rydén, A., Näslund, E., Schiller, E.M., Almgren, M., 2024. LLM-SecCode: Evaluating large language models for secure coding, in: Lecture Notes in Computer Science. Springer, pp. 100–118. doi:10.1007/978-3-031-76934-4_7.

[34] Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., Dolan-Gavitt, B., 2023. Lost at C: A user study on the security implications of large language model code assistants, in: Proceedings of the 32nd USENIX Security Symposium, USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval.

[35] Schreiber, M., Tippe, P., 2026. Security vulnerabilities in AI-generated code: A large-scale analysis of public GitHub repositories, in: Information and Communications Security (ICICS 2025), Springer, Singapore. doi:10.1007/978-981-95-3537-8_9.

[36] Semgrep, 2026. Semgrep: Lightweight static analysis for security and code quality. URL: https://semgrep.dev.

[37] Shukla, S., Joshi, H., Syed, R., 2025. Security degradation in iterative AI code generation: A systematic analysis of the paradox, in: Proceedings of the IEEE International Symposium on Technology and Society (ISTAS 2025), IEEE.

[38] Siddiq, M.L., Majumder, S.H., Mim, M.R., Jajodia, S., Santos, J.C.S., 2022. An empirical study of code smells in transformer-based code generation techniques, in: 2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM), IEEE, Limassol, Cyprus. pp. 71–82. doi:10.1109/SCAM55253.2022.00014.

[39] Snyk, 2026. Snyk: Developer-first security platform. URL: https://snyk.io.

[40] SonarSource, 2026. Sonarcloud: Continuous code quality and security. URL: https://sonarcloud.io.

[41] Sultana, S., Afreen, S., Eisty, N.U., 2026. LLMs in code vulnerability analysis: A proof of concept. URL: https://arxiv.org/abs/2601.08691. arXiv preprint arXiv:2601.08691.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361

[42] Suresh, S., Ahmed, C.M., Hassan, N.U., Ferrag, M.A., Debbah, M.A., 2026. Security weaknesses of code generated by generative AI, in: Nowroozi, E., Taheri, R., Cordeiro, L. (Eds.), Adversarial Example Detection and Mitigation Using Machine Learning. Springer, Cham. doi:10.1007/978-3-031-99447-0_6.

[43] The MITRE Corporation, 2026. CWE – Common Weakness Enumeration. URL: https://cwe.mitre.org/.

[44] Tihanyi, N., Bisztray, T., Ferrag, M.A., Jain, R., Cordeiro, L.C., 2024. How secure is AI-generated code: A large-scale comparison of large language models. Empirical Software Engineering 30. doi:10.1007/s10664-024-10590-1.

[45] Tihanyi, N., Bisztray, T., Jain, R., Ferrag, M.A., Cordeiro, L.C., Mavroeidis, V., 2023. The FormAI dataset: Generative AI in software security through the lens of formal verification, in: Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '23), ACM. doi:10.1145/3617555.3617874.

[46] Ullah, S., Han, M., Pujar, S., Pearce, H., Coskun, A., Stringhini, G., 2024. LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks, in: 2024 IEEE Symposium on Security and Privacy (SP), IEEE, San Francisco, CA, USA. pp. 862–880. doi:10.1109/SP54263.2024.00210.

[47] Wang, P., Liu, X., Xiao, C., 2025. CVE-Bench: Benchmarking LLM-based software engineering agents' ability to repair real-world CVE vulnerabilities, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2025).

[48] Yan, H., Vaidya, T., et al., 2025. Guiding AI to fix its own flaws: An empirical study on LLM-driven secure code generation. URL: https://arxiv.org/abs/2506.23034. arXiv preprint arXiv:2506.23034.

[49] Yıldırım, R., Aydın, K., Çetin, O., 2024. Evaluating the impact of conventional code analysis against large language models in API vulnerability detection, in: Proceedings of the 2024 European Interdisciplinary Cybersecurity Conference (EICC '24), ACM. pp. 57–64. doi:10.1145/3655693.3655701.

[50] Yu, X., Liu, L., Hu, X., Keung, J.W., Liu, J., Xia, X., 2024. Fight fire with fire: How much can we trust ChatGPT on source code-related tasks? IEEE Transactions on Software Engineering 50, 3435–3453. doi:10.1109/TSE.2024.3492204.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6968361